Universidad Nacional Abierta y a Distancia

Vicerrectoría Académica y de Investigación

Unidad gestora: Escuela de Ciencias Básicas Tecnología e Ingeniería ECBTI

Programa: Ciencias Básicas

Curso: Big Data

Código: 202016911A_2204

Tarea 3: Procesamiento De Datos En Tiempo Real Y Por Lotes (Batch Y Streaming)

Estudiante: Rubén Alejandro Gamba Piedrahita

Director : Primo Martin Pino Jiménez

Cali, Colombia

Octubre de 2026

INTRODUCCIÓN

El crecimiento exponencial de los volúmenes de información en las organizaciones modernas exige la implementación de arquitecturas Big Data capaces de combinar el procesamiento analítico por lotes (*Batch Processing*) con la analítica de flujos continuos en tiempo real (*Near Real-Time Streaming*).

El presente trabajo aborda de manera práctica la integración de tecnologías clave del ecosistema Big Data como **Apache Spark (PySpark)** y **Apache Kafka**. En primer lugar, se desarrolla un análisis Batch enfocado en el registro histórico de infracciones de tránsito e impacto económico en Colombia, aplicando transformaciones con DataFrames, consultas relacionales en Spark SQL y procesamiento paralelo con RDDs. En segundo lugar, se implementa una arquitectura de transmisión distribuida mediante un clúster de Apache Kafka que ingesta transacciones financieras en vivo, consumidas y procesadas en ventanas de tiempo por **Spark Structured Streaming**.

OBJETIVOS

Objetivo General

Diseñar, configurar e implementar una solución de procesamiento de datos por lotes y en tiempo real sobre una infraestructura virtualizada, utilizando PySpark, Spark SQL y Apache Kafka para la analítica avanzada de Big Data.

Objetivos Específicos

1. Desarrollar la fundamentación teórica comparativa entre Hadoop y Spark, la abstracción de RDDs/DataFrames y la arquitectura de Apache Kafka.
2. Implementar un script en PySpark para ejecutar transformaciones, consultas relacionales (Spark SQL) y agregaciones con RDDs sobre un conjunto de datos de infracciones de tránsito.
3. Desplegar un clúster local de Apache Kafka (ZooKeeper + Broker) para la generación continua de eventos de transacciones financieras en formato JSON.
4. Consumir el flujo de datos mediante Spark Structured Streaming para realizar cálculos acumulativos y promedios en tiempo real.

COMPONENTE TEÓRICO (ACTIVIDADES DE INVESTIGACIÓN)

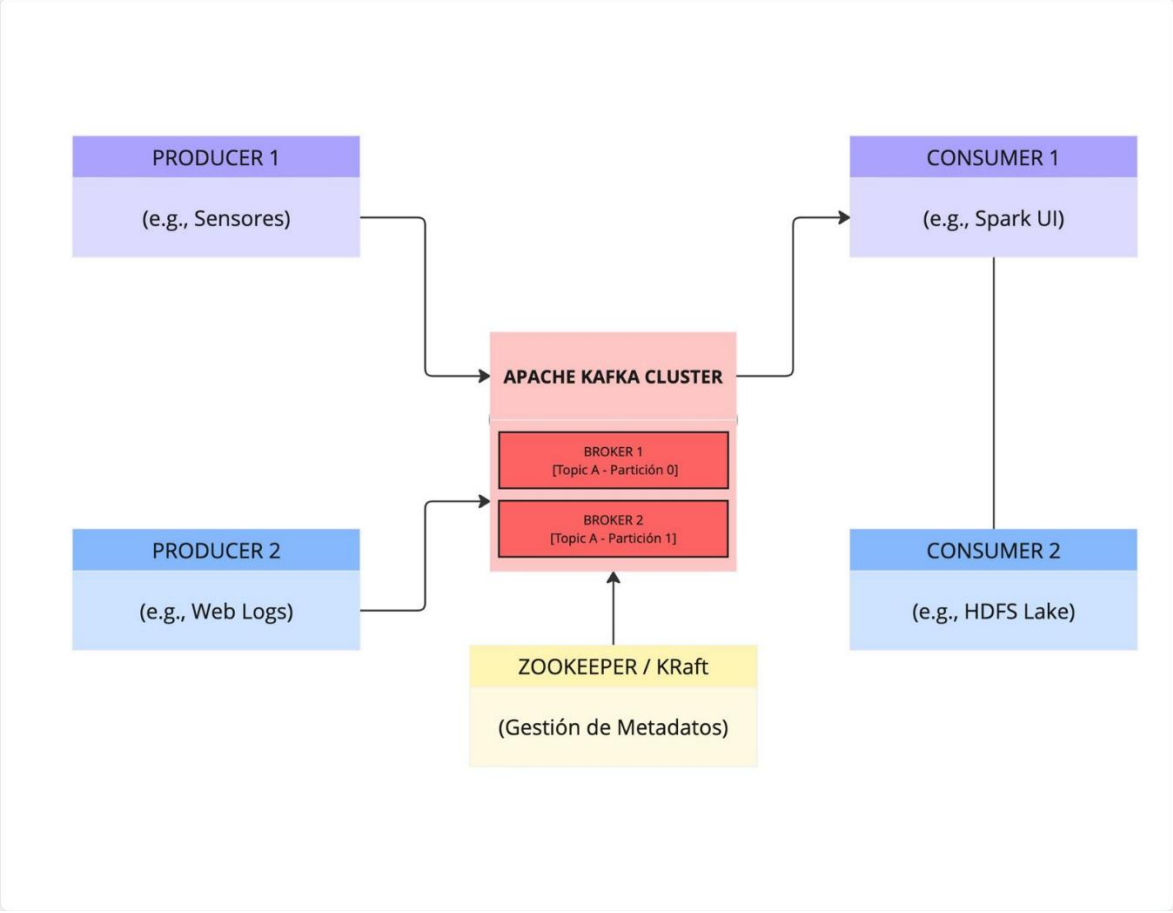
Tabla Comparativa: Apache Hadoop vs. Apache Spark

Criterio de Comparación	Apache Hadoop	Apache Spark
Arquitectura	Modelo maestro-esclavo compuesto por HDFS (almacenamiento), YARN (gestión de recursos) y MapReduce (procesamiento distribuido). Requiere I/O constante en disco.	Arquitectura distribuida basada en un Driver Program (SparkContext) y Executors. Procesa en memoria RAM mediante RDDs y DataFrames.
Tipo de Procesamiento	Diseñado exclusivamente para procesamiento por lotes (Batch) masivo sin soporte nativo en tiempo real.	Plataforma unificada: soporta Batch, Streaming en tiempo real, Spark SQL, MLlib (Machine Learning) y GraphX.
Rendimiento	Latencia alta por escrituras intermedias en disco duro durante la ejecución de tareas MapReduce.	Hasta 100 veces más rápido en memoria RAM y 10 veces más rápido en disco gracias al motor DAG (Directed Acyclic Graph).
Casos de Uso	Almacenamiento masivo histórico a bajo costo (Data Lakes HDFS) y consolidaciones nocturnas.	Análisis en tiempo real, detección de fraudes, algoritmos de Machine Learning y analítica interactiva sobre Big Data.

Conceptos de Spark: RDDs, Operaciones y DataFrames

1. **RDD (Resilient Distributed Dataset):** Abstracción fundamental de Spark. Es una colección inmutable de objetos distribuida entre los nodos del clúster. Destaca por su **particionamiento paralelo**, **inmutabilidad** y **resiliencia** (tolerancia a fallos mediante el rastreo de su grafo de linaje).
2. **Transformaciones vs. Acciones:**
    - **Transformaciones (map, filter, reduceByKey):** Operaciones perezosas (*Lazy Evaluation*) que generan un nuevo RDD sin ejecutar el cómputo de inmediato.
    - **Acciones (collect, count, take):** Desencadenan la ejecución real del DAG y devuelven un resultado al programa *Driver* o lo persisten en almacenamiento.
3. **DataFrames:** Abstracción tabular con columnas nombradas. Optimiza la memoria fuera de la pila Java (*Off-Heap*) con el motor **Tungsten** y optimiza planes de ejecución con **Catalyst Optimizer**.

Arquitectura General de Apache Kafka



DESARROLLO PRÁCTICO E IMPLEMENTACIÓN TÉCNICA

Módulo Batch Processing (PySpark y Spark SQL)

- Script Desarrollado: tarea3_batch.py
- Entorno de Ejecución: Ubuntu VM (IP: 192.168.1.13) vía SSH.

Código Fuente (tarea3_batch.py):

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum as _sum, avg, count, when
spark = SparkSession.builder \
    .appName("Analisis_Infracciones_Transito_Batch") \
    .getOrCreate()
spark.sparkContext.setLogLevel("WARN")

# Cargar Dataset
df = spark.read.csv("infracciones.csv", header=True, inferSchema=True)

# Limpieza y Columnas Derivadas
df_clean = df.filter(col("valor_multa").isNotNull()) \
    .withColumn("valor_multa", col("valor_multa").cast("double")) \
    .withColumn("categoria_gravedad",
                when(col("valor_multa") >= 1000000, "Alta")
                .when(col("valor_multa") >= 400000, "Media")
                .otherwise("Baja"))

# Consulta en Spark SQL
df_clean.createOrReplaceTempView("vista_infracciones")
query_sql = """
    SELECT tipo_vehiculo,
           COUNT(infraccion_id) as cantidad_multas,
           SUM(valor_multa) as recaudo_total,
           AVG(valor_multa) as promedio_multa
    FROM vista_infracciones
    GROUP BY tipo_vehiculo
    ORDER BY recaudo_total DESC
"""
spark.sql(query_sql).show(truncate=False)
# Operación con RDD
rdd_vehiculos = df_clean.rdd.map(lambda row: (row["tipo_vehiculo"], 1)) \
    .reduceByKey(lambda a, b: a + b)
for vehiculo, total in rdd_vehiculos.collect():
    print(f"Tipo de Vehículo: {vehiculo:<15} | Total Infracciones: {total}")
spark.stop()
```

Evidencia de Ejecución Batch:

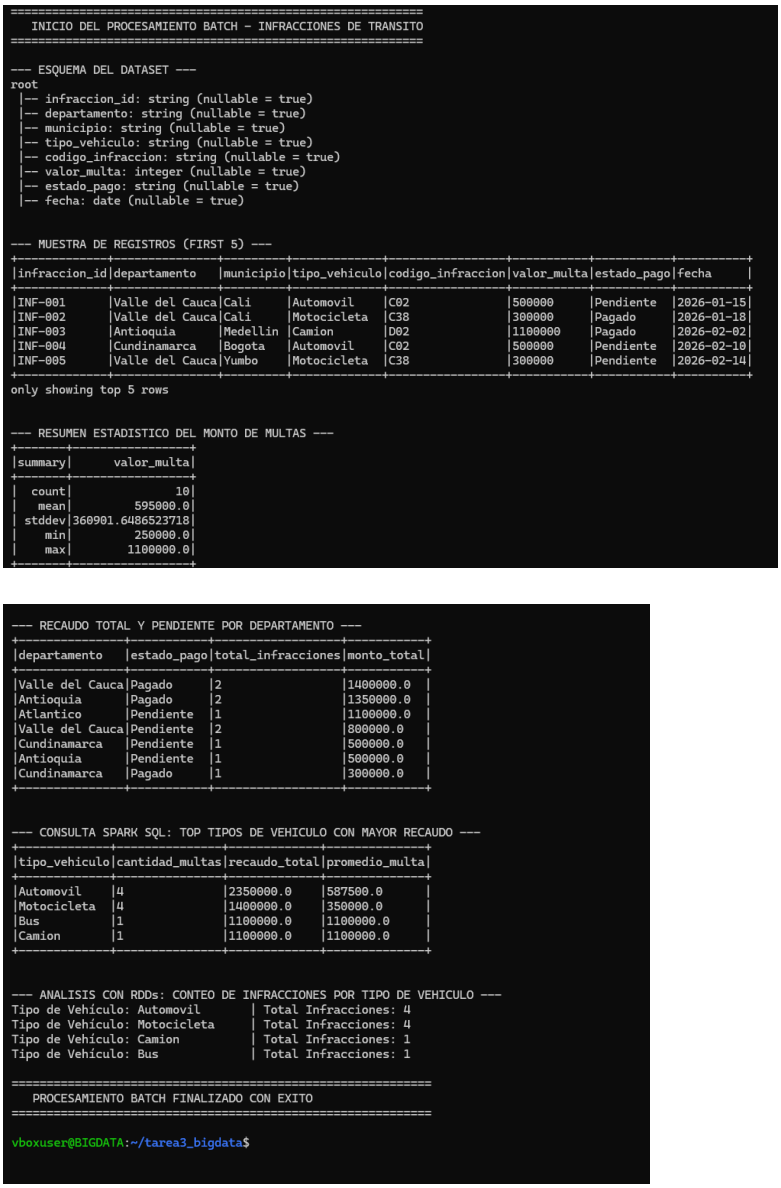


Figura 1. *Ejecución exitosa del script tarea3_batch.py en PySpark. Se evidencia la lectura del dataset de infracciones, la agregación relacional con Spark SQL por tipo de vehículo y el procesamiento distribuido mediante RDDs (map + reduceByKey).*

Módulo Near Real-Time Processing (Apache Kafka + Spark Streaming)

- Scripts Desarrollados: kafka_producer.py y spark_streaming_consumer.py

Código del Productor Kafka (kafka_producer.py):

```
import json, time, random
from datetime import datetime
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

ciudades = ['Cali', 'Bogota', 'Medellin', 'Barranquilla', 'Yumbo']
medios_pago = ['PSE', 'Tarjeta_Credito', 'Tarjeta_Debito', 'Efectivo']
transaccion_id = 1000

while True:
    transaccion_id += 1
    data = {
        'transaccion_id': f"TX-{transaccion_id}",
        'monto': round(random.uniform(10000, 250000), 2),
        'medio_pago': random.choice(medios_pago),
        'ciudad': random.choice(ciudades),
        'timestamp': datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    }
    producer.send('transacciones_topic', value=data)
    print(f"[ENVIADO] ID: {data['transaccion_id']} | Monto: ${data['monto']:,.2f} |
Ciudad: {data['ciudad']}")
    time.sleep(2)
```

Código del Consumidor Spark Streaming (spark_streaming_consumer.py):

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import from_json, col, sum as _sum, count, avg

from pyspark.sql.types import StructType, StructField, StringType, DoubleType
```

```
spark =
SparkSession.builder.appName("Monitoreo_Transacciones_Streaming").getOrCre
ate()

spark.sparkContext.setLogLevel("WARN")


schema = StructType([

    StructField("transaccion_id", StringType(), True),

    StructField("monto", DoubleType(), True),

    StructField("medio_pago", StringType(), True),

    StructField("ciudad", StringType(), True),

    StructField("timestamp", StringType(), True)

])


df_kafka = spark.readStream \

    .format("kafka") \

    .option("kafka.bootstrap.servers", "localhost:9092") \

    .option("subscribe", "transacciones_topic") \

    .option("startingOffsets", "latest") \

    .load()

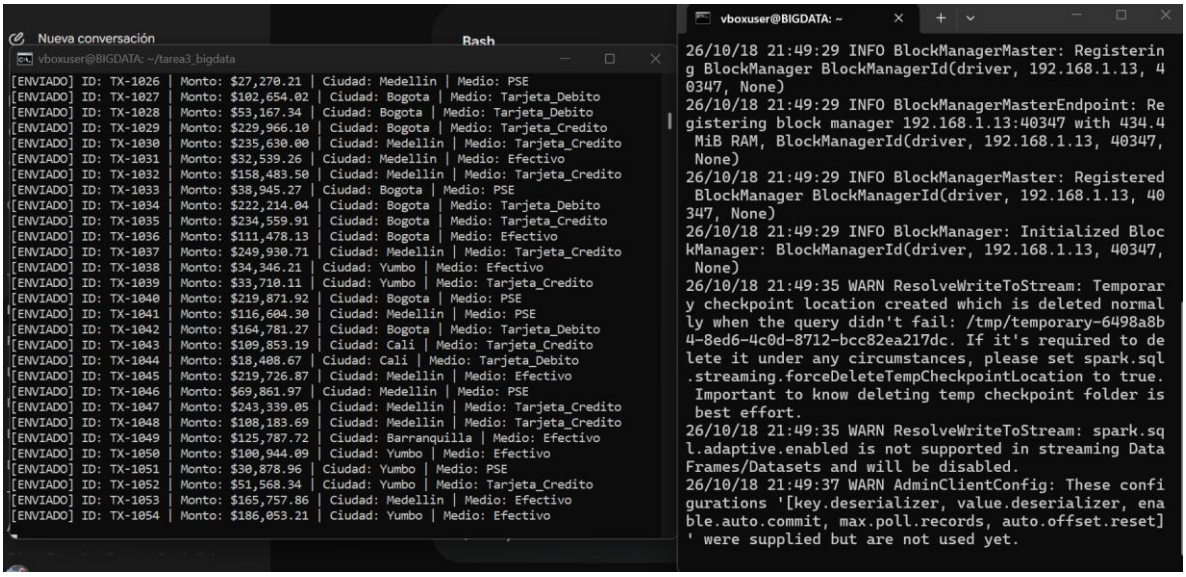

df_parsed = df_kafka.selectExpr("CAST(value AS STRING) as json_payload") \

    .select(from_json(col("json_payload"), schema).alias("data")) \

    .select("data.*")
```

```
df_metrics = df_parsed.groupBy("ciudad").agg(
    count("transaccion_id").alias("total_transacciones"),
    _sum("monto").alias("monto_acumulado"),
    avg("monto").alias("monto_promedio")
)
```

```
query = df_metrics.writeStream.outputMode("complete").format("console").start()
query.awaitTermination()
```



Link del video sustentación: https://youtu.be/MsTaq2LkLZY

CONCLUSIONES

1. La evaluación práctica demuestra la clara superioridad de Apache Spark sobre MapReduce en términos de velocidad e integrabilidad, permitiendo procesar conjuntos de datos tabulares mediante DataFrames y Spark SQL de forma limpia y eficiente.
2. La combinación de Apache Kafka como *message broker* distribuido con Spark Structured Streaming constituye una arquitectura robusta de baja latencia, ideal para escenarios analíticos críticos como el monitoreo de fraudes o transacciones en tiempo real.
3. El uso de RDDs dentro de PySpark ofrece un control de muy bajo nivel para operaciones personalizadas (map y reduceByKey), aunque la capa de DataFrames resulta superior en rendimiento al aprovechar el optimizador Catalyst.

BIBLIOGRAFÍA (Norma APA)

- Apache Software Foundation. (2024). *Apache Spark Documentation: Structured Streaming Programming Guide*. https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html
- Apache Software Foundation. (2024). *Apache Kafka Documentation: Core Concepts & Architecture*. https://kafka.apache.org/documentation/
- Jaramillo, M. L. (2024). *Fundamentos de Big Data y Arquitecturas Distribuidas*. Universidad Nacional Abierta y a Distancia (UNAD).